# Software Details

## SOFTWARE OVERVIEW

The code controls a four-wheeled car that can perform certain commands like automatically avoiding obstacles in its path or can be driven manually. It runs on MicroPython, every time the ESP32 powers on, MicroPython automatically runs the main file called main.py which has all this commands and also enables Bluetooth advertising. The user controls the car through an app built in MIT App Inventor, which sends short text commands to the ESP32 over Bluetooth Low Energy (BLE) when the buttons on the app are pressed.

The two main platforms used are MicroPython for the ESP32 code and MIT App Inventor for the Android app. The circuit was planned using Cirkit Designer, the code was written and uploaded using Thonny IDE, and everything is stored on GitHub including the documentation of the process.

## SYSTEM LOGIC

### STARTUP SEQUENCE:

- Physical switch powers the ESP32 that runs the MicroPython code (main.py) automatically
- BLE radio turns on and ESP32 starts advertising as "ESP32-NDtV"
- Main while True loop starts with auto_mode set to False (manual mode by default)
- Scan button on the app is pressed and "ESP32-NDtV" can be seen on the list display
- Connect to ESP32-NDtV to access the three modes
- Loop runs every 30ms and checks if a command has arrived over BLE

### MANUAL MODE:

- Up arrow pressed → app sends "F" → forward() function runs → set_speed(1023, 1023) function applies full PWM to all enabled pins → all 4 wheels spin forward → NeoPixel turns green
- Down arrow pressed → app sends "B" → backward() function runs → set_speed(1023, 1023) function applies full PWM to all enabled pins → all 4 wheels spin backward → NeoPixel turns green
- Left arrow pressed → app sends "L" → left() function runs and left wheel move backwards and right wheels move forward → set_speed(1023, 200) function slows the left side to 200 duty while right stays at 1023 → car curves left → NeoPixel turns green
- Right arrow pressed → app sends "R" → right() function runs and right wheel move backwards and left wheels move forward → set_speed(200, 1023) function slows the right side to 200 duty while left stays at 1023 → car curves right → NeoPixel turns green
- Stop pressed → app sends "S" → stop() function runs → set_speed(0, 0) cuts all PWM → car comes to a stop → NeoPixel turns off
- If "auto" is received → auto_mode set to True → next loop cycle switches to auto mode branch

### AUTO MODE:

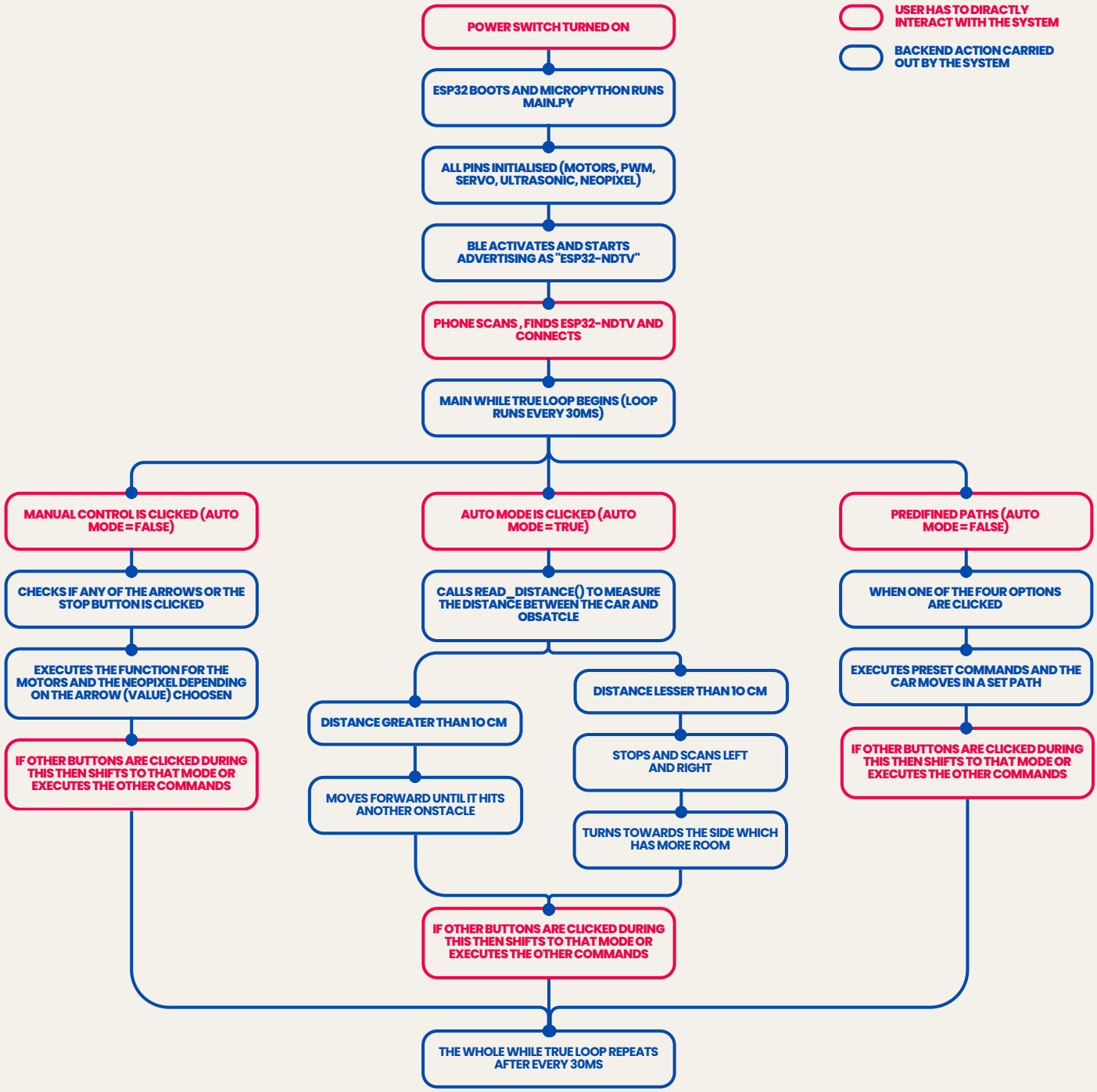
- Loop no longer waits for BLE commands and reads the sensor inputs every cycle instead
- read_distance() function fires a 10 microsecond pulse from the TRIG pin and time_pulse_us() function waits for the ECHO pin with a 30ms timeout
- If no echo received (object too far or sensor error) → returns 250cm safely using try/except to avoid any errors that might stop the whole code
- If echo received → duration divided by 58 → gives distance in centimetres → and returns this distance
- Distance greater than 10cm → path is clear → forward() function runs → car moves forward→ NeoPixel glows green
- Distance 10cm or less → obstacle is detected → avoid_obstacle() function is called → NeoPixel glows red → car stops and moves back

- Inside avoid_obstacle() → NeoPixel is orange while scanning the surrounding → the servo turns the ultrasonic sensor left and right → distance is recorded and compared → car turns toward whichever side has more room → NeoPixel off → loop resumes
- If any direction manual command arrives during auto mode → auto_mode set to False → returns to manual mode on next cycle

## PREDEFINED PATH MODE:

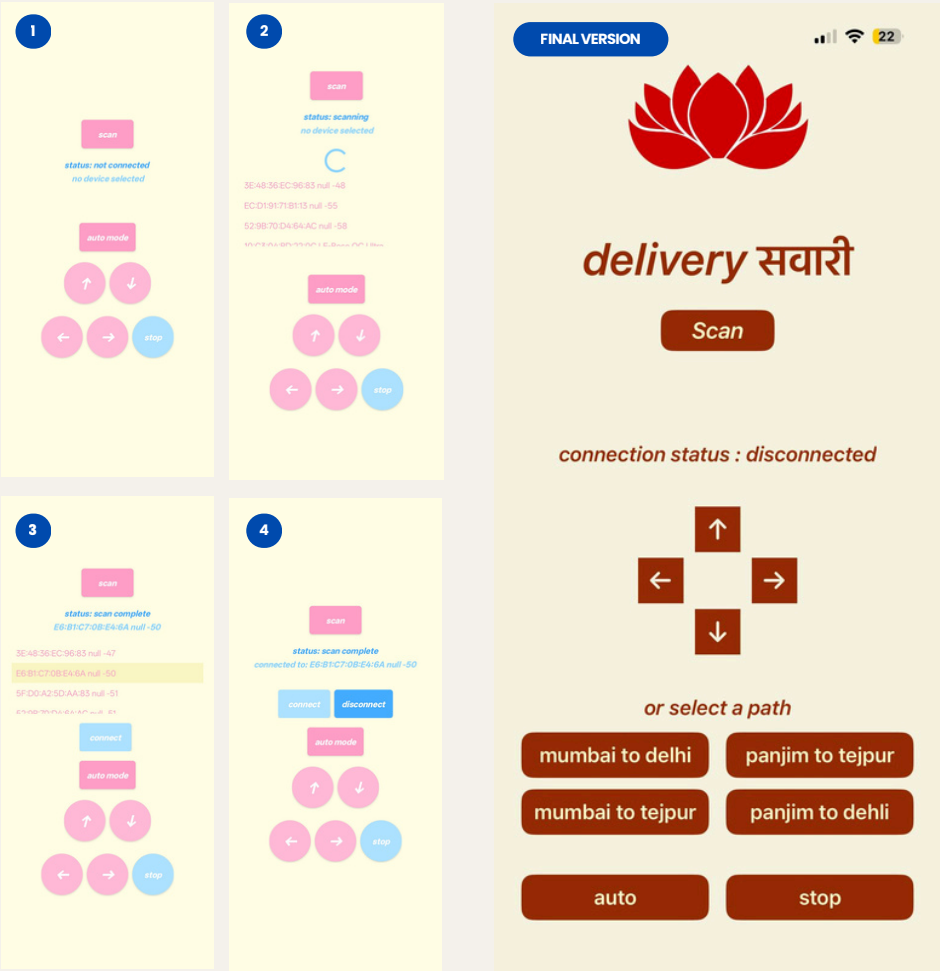
- App sends a route command → ESP32 calls the matching path function
- selecting Mumbai to Dehli runs path_md() → 2m straight
- selecting Mumbai to Panjim runs path_mp() → 1m straight
- selecting Panjim to Dehli runs path_pd() → 1 m straight, 1m left, 1 m straight
- selecting Mumbai to Tezpur runs path_mt() → 1m straight, 1m right, 1 m straight

## FLOW CHARTS



# APP FLOW (MIT APP INVENTOR)

## SCREEN FLOW



## BUTTON ACTIONS

- Scan button - searches for nearby BLE devices
- Dropdown list - shows devices found nearby
- Connect button - connects to selected device
- Disconnect button - disconnects from current device
- Up arrow – moves forward
- Left arrow – moves to left
- Right arrow - moves to right
- Down arrow - moves backwards
- Mumbai to Dehli - 2m straight
- Mumbai to Panjim - 1m straight
- Panjim to Dehli - 1 m straight, 1m left, 1 m straight
- Mumbai to Tezpur - 1m straight, 1m right, 1 m straight
- Auto button - switches car to obstacle avoidance mode
- Stop button - stops all movement immediately

## DATA FLOW

- Every button has a TouchDown event in App Inventor
- When tapped, the event calls BluetoothLE1.WriteBytes
- This sends a short text string wirelessly over BLE to the ESP32
- The ESP32's BLE interrupt receives it within milliseconds
- The string is decoded and stored in the variable called value
- The main loop picks it up on the next 30ms cycle and acts on it
- According to the value received, functions are executed to that corresponding value